KWT

KDL We Token

Compact, Hardened uthentication Protocol

Technical White Paper | Draft 1.0

KWT Working Group
pril 13, 2026

Contents

1 bstract

JSON Web Tokens (JWT) are the dominant authentication primitive on the web. They are flexible, widely supported, and well-understood. They are also verbose, structurally under-constrained, and carry a long history of catastrophic implementation vulnerabilities rooted directly in that flexibility.

This paper introduces **KWT** (KDL Web Token), a protocol that replaces JWT's JSON payload encoding with KDL (KDL Document Language), couples it with a versioned, algorithm-fixed cryptographic envelope, and defines a canonical binary serialization that preserves KDL's information density advantage through the encryption boundary.

The central thesis is that *information density* — defined as meaningful bits of claim data per byte of wire transmission — is a measurable, compoundable engineering property. KWT achieves 2.5–4× the information density of JWT for equivalent payloads, translating directly into reduced storage costs, lower transmission latency, less CPU time for cryptographic operations, and lower energy expenditure at scale.

We derive the mathematical basis for these claims, specify the complete protocol, and provide a reference implementation in Rust.

2 Introduction and Motivation

2.1 The JWT Landscape

JWT (RFC 7519) has become ubiquitous in web authentication, PI authorization, and session management. The format encodes a header, payload, and signature as three base64url-encoded segments separated by periods:

```
base64url(header).base64url(payload).base64url(signature)
```

JWT's popularity is justified: the format is human-readable before encoding, self-contained, and stateless. However, the specification's emphasis on flexibility has produced a well-documented catalogue of vulnerabilities:

- The `alg:none` attack — removing the signature by declaring no algorithm is in use
- lgorithm confusion — confusing HM C and RS families to forge signatures with public keys
- `kid` header injection — passing a key identifier to a database query unsanitized
- Weak secret keys — brute-forcing `HS256` with a short secret offline against intercepted tokens

These are not implementation bugs. They are the natural consequence of a format that lets the token itself negotiate its own security parameters — a fundamentally unsafe design choice.

2.2 The Size Problem

Beyond security, JWT carries a structural inefficiency that compounds at scale. representative JWT for a typical web session:

```
{"alg":"RS256","typ":"JWT"}
.{"sub":"user_882","iat":1744459200,"exp":1744545600,
  "roles":["admin","editor"],"aud":"api.example.com"}
.MEUCIQDd3...{86 bytes of base64 signature}
```

The encoded payload alone is approximately 160–180 bytes. t 100 million authentications per day — a modest figure for a production service — this represents roughly 17 GB of payload data transmitted daily, excluding headers and signature. Every byte stored in session databases, written to audit logs, and processed by middleware multiplies the cost.

2.3 The KDL Opportunity

KDL (KDL Document Language, pronounced "Cuddle") is a human-readable document language designed to eliminate the structural noise of JSON while preserving its semantic expressiveness. The same payload in KDL:

```
subject "user_882"
issued 1744459200
expires 1744545600
 udience "api.example.com"
roles "admin" "editor"
```

This is 91 characters versus JSON's approximately 115 characters — a 21% raw reduction before any encoding. More importantly, KDL's structure maps cleanly to a compact canonical binary representation, allowing the density advantage to survive encryption.

3 Information Density: The Calculus

3.1 Defining Information Density

We define information density D for a token format as the ratio of semantic payload bits S to total transmitted bytes W:

$$D = \frac{S}{W} \tag{1}$$

where S is the count of bits required to represent the minimum lossless encoding of all claim values (excluding field names, which are structural overhead), and W is the total wire byte count of the transmitted token. higher D means more claim data per transmitted byte.

3.2 Structural Overhead nalysis

Consider a canonical benchmark payload with five claims:

Table 1: Semantic bit content of benchmark payload

Claim	Value	Semantic bits	Encoding note
`subject`	user_882	∼40 bits	Integer user ID space
`issued`	1744459200	32 bits	Unix timestamp
`expires`	1744545600	32 bits	Unix timestamp
`audience`	api.example.com	∼112 bits	UTF-8 string
`roles`	[admin, editor]	∼16 bits	2-bit enum × 2

Total semantic content: approximately **232 bits (29 bytes)**. Everything else is structural overhead: field names, delimiters, encoding artefacts.

3.3 Per-Format Overhead Breakdown

3.3.1 JSON Encoding

```
{"sub":"user_882","iat":1744459200,"exp":1744545600,
 "aud":"api.example.com","roles":["admin","editor"]}
```

Raw byte count: 112 bytes. fter base64url encoding: **150 bytes**. Structural overhead is 61 bytes (54%).

```
D(JWT-JSON) = 232 bits / 150 bytes = 1.55 bits/byte
```

3.3.2 KDL Text Encoding

Raw KDL: 91 bytes. fter base64url encoding: **122 bytes**.

```
D(KWT-text) = 232 bits / 122 bytes = 1.90 bits/byte
```

3.3.3 KWT Canonical Binary Encoding

KWT's key innovation is a canonical binary serialization mapping KDL structure to 1-byte opcodes, varint-encoded lengths, and enum values. The benchmark payload encodes to **43 bytes** before encryption. fter XChaCha20-Poly1305 (which appends only a fixed 16-byte authentication tag) and base64url encoding: approximately **79 bytes** for the ciphertext segment. Combined with nonce and version prefix, total token: **∼105–120 bytes**.

```
D(KWT-binary) = 232 bits / 79 bytes = 2.94 bits/byte
```

3.4 Comparative Summary

Table 2: Information density comparison across token formats

Format	Pre-encrypt	Full token	Density (bits/byte)
JWT (JSON, RS256)	112 B	380–500 B	1.55
KWT (KDL text)	91 B	220–380 B	1.90
KWT (canonical binary)	43 B	105–160 B	**2.94**
Theoretical minimum	29 B	∼65 B	3.57

The canonical binary encoding achieves 89% of the theoretical information-density ceiling while remaining fully self-describing and verifiable.

3.5 The Compounding Cost Model

Let T be daily token volume, $B(f)$ the bytes per token for format f, C_s the cost per GB-month of storage, C_t the cost per GB of transmission, and C the CPU energy cost per byte decrypted.

$$\text{Daily savings} = T \cdot \ B(\text{JWT}) \quad B(\text{KWT})) \cdot (C_s + C_t + C\ ) \tag{2}$$

For a service at $T = 10^8$ tokens/day with $B(\text{JWT}) = 400$ bytes and $B(\text{KWT}) = 120$ bytes:

- Payload reduction: 280 bytes per token
- Daily data reduction: **28 GB/day**
- nnual data reduction: **∼10.2 TB/year**

t WS S3 standard pricing (≈$0.023/GB-month), this yields approximately $2,800/year in storage savings alone before transmission and compute are considered.

4 KWT Protocol Specification

4.1 Design Principles

KWT is designed around three non-negotiable constraints:

1. The token never negotiates its own cryptographic parameters. The version prefix determines the algorithm; the payload cannot override it.
2. The payload is always encrypted. There is no signed-but-unencrypted mode.
3. The parser is maximally strict. Unknown fields, out-of-range values, and malformed structure are hard errors, never silent ignores.

4.2 Token Structure

```
v{N}.<base64url(nonce)>.<base64url(ciphertext_with_tag)>
```

There is no header segment. The version prefix `v{N}` *is* the header — it is not attacker-controlled, is not base64-encoded, and determines the complete cryptographic profile.

4.3 Version Table

Table 3: KWT version registry (permanent; no runtime negotiation)

Ver.	Encryption	KDF	Use Case
v1	XChaCha20-Poly1305	HKDF-SH 256	Standard symmetric tokens
v2	ES-256-GCM	HKDF-SH 256	Hardware-accelerated (ES-NI); **browser-native**
v3	XChaCha20-Poly1305	X25519 + HKDF	symmetric: public encrypt, private decrypt

Browser note

v2 (ES-256-GCM) is the only version whose primitives are natively available via the W3C Web Crypto PI in all modern browsers. See Section 7 for details.

4.4 Canonical Binary Payload Format

4.4.1 Opcode Registry

ll registered claim fields are assigned a 1-byte opcode that encodes both the field identity and the value type.

Table 4: KWT opcode registry

Range	Claim	Wire encoding	Notes
`0x10-0x1F`	Subject identifiers	varint len + UTF-8	
`0x20-0x2F`	Timestamps	uint32 LE (4 B)	Unix epoch, seconds
`0x30-0x3F`	udience / issuer	varint len + UTF-8	
`0x40-0x4F`	Roles	uint8 count + uint8[]	Closed enum
`0x50-0x5F`	Scopes	uint8 count + uint8[]	Closed enum
`0x60-0x6F`	JTI / nonce	16 raw bytes	UUID v7 only
`0x70-0x7F`	Custom claims	varint len + bytes	pp-defined
`0x80`	END marker	none	Required, last byte

4.4.2 Varint Encoding

Variable-length integers use Protocol Buffers-style encoding: the low 7 bits carry data; the high bit signals continuation. Values 0–127 encode in 1 byte; 128–16 383 in 2 bytes. This is optimal for token field lengths.

4.4.3 Canonicalization Requirements

Before encryption, the binary payload *must* be serialized in canonical form:

- Fields appear in *ascending opcode order*
- No duplicate opcodes
- Strings are valid UTF-8, NFC-normalized
- Timestamps: `expires_at` > `issued_at`
- The `0x80` END marker is the final byte

4.5 Nonce Strategy

XChaCha20-Poly1305 uses a 192-bit (24-byte) nonce. KWT generates nonces using a CSPRNG. The probability of nonce collision over 2^{32} tokens under the same key is approximately 2^{-128} — well below any practical threshold.

Counter-based nonces are explicitly disallowed unless the implementation can guarantee strict monotonicity across distributed systems and process restarts.

4.6 Key Derivation

Raw symmetric keys are not used directly. KWT uses HKDF-SH 256:

$$K_{\text{session}} = \text{HKDF}(\text{IKM} = K_{\text{m ster}},\ \text{salt} = \text{nonce}[0{:}32],\ \text{info} = \texttt{"kwt-v1-encryption"}) \quad (3)$$

This ensures that even a nonce collision cannot recover the master key.

4.7 Token Validation lgorithm

Validation steps must be performed in the order below. ny failure returns a generic 401 — callers must not expose which step failed.

1. Parse version prefix; reject unknown versions immediately.
2. Decode base64url segments; reject malformed base64.
3. **Check JTI bloom filter** before performing any cryptography (prevents decryption-cost denial-of-service from replay flood).
4. Derive session key via HKDF.
5. Decrypt and authenticate (E D); reject if tag fails.
6. Parse canonical binary payload; reject malformed opcodes.
7. Validate `expires_at` > `now`.
8. Validate `audience` matches server's registered audience.
9. dd JTI to bloom filter; return validated claims.

5 Security nalysis

5.1 ttack Surface Comparison

Table 5: ttack surface comparison: JWT vs KWT

ttack Vector	JWT Exposure	KWT Exposure
`alg:none` bypass	Critical — native to spec	Impossible — no algorithm field
lgorithm confusion	High — attacker controls header	Impossible — version is immutable
`kid` injection	High — arbitrary string to DB	Mitigated — key registry by UUID
Payload disclosure	High — only signed, not encrypted	None — always E D-encrypted
Weak secret brute-force	Possible with short HS256 keys	Mitigated by HKDF derivation
Token replay	Optional (`jti`)	Required (bloom filter)
Timing side-channels	Library-dependent	Constant-time (RustCrypto)

5.2 The Encrypted Payload Security Model

Because the entire payload is encrypted, an attacker who intercepts a KWT token learns nothing beyond the token's version and length. JWT containing personally identifiable information (user IDs, roles, email) that appears in a server log or CDN access log constitutes a data exposure event. KWT in the same context discloses only a random-looking byte string.

5.3 The No-Header Guarantee

The version string `v1` is matched against a hard-coded list — it is not parsed as a data structure. n attacker cannot construct a malformed header to trigger unexpected parser behaviour: there is no header to parse.

5.4 Bloom Filter Replay Prevention

Stateless tokens are inherently replayable within their validity window. KWT mandates replay prevention via a counting bloom filter in Redis:

- JTI checked on every validation request before decryption
- TTL on filter entries matches token max TTL
- False-positive rate of 0.1% is acceptable (results in a rejected valid token, not an accepted replayed token)
- UUID v7's time-ordering allows efficient range-based eviction

6 Comparison with Existing Standards

6.1 KWT vs. JWT

Table 6: KWT vs JWT feature comparison

Property	JWT	KWT
Payload format	JSON (verbose)	Binary canonical (compact)
Encryption	Optional (JWE)	Mandatory
lgorithm agility	Full (dangerous)	None (safe)
Header	ttacker-controlled JSON	Immutable version prefix
Typical token size	300–500 bytes	100–160 bytes
Info density	∼1.55 bits/byte	∼2.94 bits/byte
Library support	Ubiquitous	Reference only (Rust)
udit history	Extensive	None (new protocol)

6.2 KWT vs. P SETO

P SETO solves the algorithm-agility problem identically: versioned tokens with fixed cryptographic profiles. KWT is best understood as P SETO with a binary-encoded KDL payload rather than JSON.

Table 7: KWT vs P SETO feature comparison

Property	P SETO	KWT
Security model	Versioned, fixed algos	Versioned, fixed algos
Payload format	JSON	Binary canonical
Typical payload	150–250 bytes	40–80 bytes
Info density	∼1.75 bits/byte	∼2.94 bits/byte
Library support	10+ languages	Rust reference only
udit status	Multiple audits	No audits

7 Browser Compatibility

7.1 Web Crypto PI Coverage

The W3C Web Crypto PI (`window.crypto.subtle`) provides the cryptographic primitives available natively in browser JavaScript. Its algorithm support determines which KWT versions can run without a userland library.

Table 8: KWT primitive availability in the Web Crypto PI

Primitive	vailable	Notes
ES-256-GCM (v2)	**Yes**	Full support Chrome, Firefox, Safari, Edge
HKDF-SH 256	**Yes**	ll major browsers
`getRandomValues`	**Yes**	ll browsers; use for nonce generation
XChaCha20-Poly1305 (v1)	**No**	Not in spec; open W3C issue #223 since 2019
ChaCha20-Poly1305 (96-bit nonce)	**No**	Used in TLS 1.3 internally but not exposed

The practical consequence: **KWT v2 (ES-256-GCM) can be implemented entirely with Web Crypto PI primitives**. KWT v1 and v3 require either a Web ssembly module or a JavaScript fallback library.

7.2 Recommended Browser Strategy

1. **Use v2 (ES-256-GCM) for all browser-issued or browser-validated tokens.** ES-256-GCM with hardware ES-NI acceleration is cryptographically equivalent to XChaCha20-Poly1305 for this use case; the nonce-reuse risk is managed by always generating nonces via `crypto.getRandomValues()`.
2. **If v1 is required** (e.g., for interoperability with a Rust backend that issued v1 tokens), ship a `libsodium-wasm` bundle (∼163 kB gzipped) or compile the reference Rust implementation to W SM via `wasm-pack`.
3. **Never place the master key in browser storage.** Browsers should receive a *derived session key* or a short-lived *wrapped key* via a secure server handshake — never the raw `MasterKey` bytes.

7.3 v2 JavaScript Reference (Web Crypto PI)

```
// KWT v2 browser validator ( ES-256-GCM + HKDF-SH 256)
// The rawKey should arrive via a secure key-agreement handshake,
// never hardcoded or stored in localStorage.

async function kwtValidate(tokenStr, rawKey, expected udience) {
  const parts = tokenStr.split('.');
  if (parts.length !== 3 || parts[0] !== 'v2') {
    throw new Error('malformed token or wrong version');
  }

  const nonce = base64urlDecode(parts[1]);  // 12 bytes for  ES-GCM
  const ciphertext = base64urlDecode(parts[2]);

  // 1. Import master key material
  const keyMaterial = await crypto.subtle.importKey(
    'raw', rawKey, 'HKDF', false, ['deriveKey']
  );

  // 2. Derive session key via HKDF-SH 256
  const sessionKey = await crypto.subtle.deriveKey(
```

```
    {
      name: 'HKDF',
      hash: 'SH -256',
      salt: nonce.slice(0, 32).padEnd(32, 0), // pad 12-byte nonce to 32
      info: new TextEncoder().encode('kwt-v2-encryption'),
    },
    keyMaterial,
    { name: ' ES-GCM', length: 256 },
    false,
    ['decrypt']
  );

  // 3. Decrypt and authenticate
  const plaintext = await crypto.subtle.decrypt(
    { name: ' ES-GCM', iv: nonce },
    sessionKey,
    ciphertext
  );

  // 4. Parse canonical binary payload
  const claims = parseKwtBinary(new Uint8 rray(plaintext));

  // 5. Validate expiry (with 30s leeway)
  const now = Math.floor(Date.now() / 1000);
  if (claims.expires t + 30 < now) throw new Error('token expired');

  // 6. Validate audience
  if (claims.audience !== expected udience) throw new Error('audience mismatch');

  return claims;
}
```

Listing 1: KWT v2 token validation in browser JavaScript

8 Implementation Guidance

8.1 Recommended Rust Dependencies

Table 9: Rust crates for KWT reference implementation

Crate	Purpose	Notes
`chacha20poly1305`	XChaCha20-Poly1305 (v1/v3)	RustCrypto; constant-time
`aes-gcm`	ES-256-GCM (v2)	Uses ES-NI where available
`hkdf` + `sha2`	HKDF-SH 256	RFC 5869 compliant
`uuid`	UUID v7 for JTI	Time-ordered for bloom filter
`rand`	CSPRNG nonce generation	OS entropy source
`zeroize`	Key material erasure	Prevents key in swap/core dump

8.2 Critical Implementation Notes

- **Zeroize key material.** Rust does not guarantee memory erasure on drop. Use the `zeroize` crate explicitly on all key buffers.
- **Fuzz-test the canonical codec.** Serialize-deserialize round-trip fuzzing with `cargo-fuzz` must be completed before deployment.
- **Bloom filter eviction must be load-tested.** n unbounded bloom filter will eventually produce an unacceptable false-positive rate.
- **llow clock leeway.** Service-to-service clock skew can cause valid tokens to appear expired. configurable leeway of 30–60 seconds is recommended.
- **Maintain validation order.** The bloom filter check must precede decryption to prevent replay-flood denial-of-service.

8.3 Migration Path from JWT

Migration proceeds in three phases:

1. **Dual-accept.** The server accepts both JWT and KWT. New tokens are issued as KWT. Old JWT sessions expire naturally over the TTL window.
2. **KWT-only.** fter the dual-accept window, JWT acceptance is removed. ll clients must be issuing and receiving KWT.
3. **Cleanup.** JWT parsing code is removed; JWT secret keys are rotated out and destroyed.

9 Conclusion

KWT demonstrates that token format is a measurable engineering variable with direct cost implications, not merely a cosmetic choice. By replacing JSON's verbose, delimiter-heavy encoding with KDL's node-structured model and a compact canonical binary serialization, KWT achieves approximately **2.94 bits of semantic information per transmitted byte** — compared to JWT's 1.55 bits/byte — a 90% improvement in information density.

This density advantage compounds across storage, transmission, and compute dimensions. t scale, the savings are material. t the protocol level, the versioned, algorithm-fixed design eliminates the entire class of flexibility-based JWT attacks that have dominated web security incident reports for a decade.

The primary limitation of KWT at this stage is the absence of battle-tested implementations and independent security audits. The recommendation is to treat this specification as a design target: implement the reference Rust library, subject it to fuzz testing and professional audit, and deploy behind a feature flag in a high-throughput non-critical path before promoting to primary authentication infrastructure.

References

[1] M. Jones, J. Bradley, N. Sakimura. *RFC 7519: JSON Web Token (JWT)*. IETF, May 2015.

[2] S. rciszewski. *P SETO: Platform- gnostic Security Tokens*, v4 spec, 2022.

[3] D. J. Bernstein. *ChaCha, a variant of Salsa20*. S SC 2008.

[4] KDL Document Language Specification. https://kdl.dev, 2024.

[5] H. Krawczyk, P. Eronen. *RFC 5869: HM C-based Key Derivation Function (HKDF).* IETF, 2010.

[6] W3C. *Web Cryptography PI*, W3C Recommendation. https://www.w3.org/TR/WebCrypto PI/

[7] W3C WebCrypto GitHub Issue #223. *Please add support for chacha20-poly1305.* https://github.com/w3c/webcrypto/issues/223

[8] MDN Web Docs. *SubtleCrypto.deriveKey().* https://developer.mozilla.org/en-US/docs/Web/ PI/SubtleCrypto/deriveKey